

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP DOANH NGHIỆP

NGÀNH: CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG TRUY
XUẤT TIN NHẮN TIẾNG VIỆT THỜI GIAN THỰC DỰA TRÊN
CỐC CỐC TOKENIZER VÀ ELASTICSEARCH**

Cán bộ hướng dẫn: Trần Nhật Anh

Giảng viên đánh giá: Đặng Minh Công

Sinh viên thực hiện: Lê Tuấn Cảnh

Mã sinh viên: 23020013

Lớp: K68I-IT1

Hà Nội, tháng 10 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin được gửi lời cảm ơn chân thành và sâu sắc nhất đến Ban Giám hiệu Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội, cùng các thầy cô giáo trong Khoa Công nghệ Thông tin đã tận tình giảng dạy, trang bị cho em những kiến thức nền tảng khoa học máy tính quý báu trong suốt quá trình học tập tại trường. Đặc biệt, em xin gửi lời tri ân sâu sắc đến thầy Đặng Minh Công – giảng viên đánh giá học phần đã theo sát, định hướng phương pháp luận khoa học và dành cho em những lời khuyên sâu sắc để hoàn thiện bản báo cáo này.

Em xin trân trọng cảm ơn Ban tổ chức Chương trình Đào tạo Nhân tài AI Thực chiến do Trường Đại học VinUniversity tổ chức. Trong 12 tuần tham gia kỳ thực tập hè, 6 tuần học tập chuyên sâu đầu tiên tại VinUniversity đã trang bị cho em hệ thống kiến thức toàn diện và nền tảng kỹ thuật vững vàng về trí tuệ nhân tạo, xử lý dữ liệu lớn và tư duy thiết kế phần mềm hiện đại.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới Ban Lãnh đạo Công ty Cổ phần Vin Smart Future, cùng toàn thể các anh chị kỹ sư tại Khối Chat – Ban Phát triển Sản phẩm đã tạo điều kiện thuận lợi nhất để em được tham gia 6 tuần thực chiến trực tiếp tại doanh nghiệp. Em xin gửi lời cảm ơn chân thành nhất tới anh Trần Nhật Anh – Kỹ sư Backend Cấp cao tại Khối Chat, người đã trực tiếp kèm cặp 1-on-1, định hướng công nghệ và chia sẻ những kinh nghiệm thực tiễn quý báu, giúp em tự tin vượt qua nhiều rào cản kỹ thuật phức tạp để hoàn thành tốt mục tiêu nghiên cứu và xây dựng thành công hệ thống của đề tài.

Mặc dù em đã nỗ lực hết mình để hoàn thành khối lượng công việc, song do giới hạn về mặt thời gian và kinh nghiệm thực tế, bản báo cáo khó tránh khỏi những điểm còn thiếu sót. Em rất mong nhận được sự góp ý, chỉ bảo của các thầy cô và các anh chị chuyên gia để đề tài nghiên cứu được hoàn thiện hơn nữa.

Em xin chân thành cảm ơn!

TÓM TẮT BÁO CÁO

Tóm tắt tiếng Việt

Truy xuất tin nhắn hội thoại thời gian thực (Real-time Conversational Message Retrieval) là một thành phần cốt lõi trong các nền tảng nhắn tin tức thời quy mô lớn, đòi hỏi đồng thời độ trễ truy vấn cực thấp, khả năng đánh chỉ mục luồng dữ liệu liên tục với thông lượng cao và độ chính xác xếp hạng vượt trội. Thách thức lớn nhất đối với tiếng Việt là đặc trưng của ngôn ngữ đơn lập với hiện tượng nhập nhằng ranh giới từ vựng và bất từ ghép, khiến các công cụ tìm kiếm chuẩn phương Tây suy giảm nghiêm trọng độ chính xác do phân mảnh từ ghép thành các âm tiết rời rạc. Bên cạnh đó, thói quen gõ phím nhanh không dấu và gõ dở từng ký tự của người dùng đặt ra bài toán phức tạp về mặt xếp hạng.

Báo cáo này trình bày toàn bộ quá trình nghiên cứu, thiết kế kiến trúc và phát triển hệ thống truy xuất tin nhắn tiếng Việt thời gian thực mà em đã thực hiện trong 12 tuần thực tập (gồm 6 tuần đào tạo chuyên sâu tại VinUniversity và 6 tuần thực chiến tại Khối Chat – Công ty Cổ phần Vin Smart Future). Hệ thống kết hợp hài hòa ba thành phần trọng tâm: Động cơ phân tích từ vựng Cốc Cốc Tokenizer tích hợp qua lớp cầu nối CGO an toàn bộ nhớ không rò rỉ RAM; Cụm tìm kiếm phân tán Elasticsearch; và một Động cơ Lưu trữ Nhị phân Chuyên dụng độc lập do em tự thiết kế dựa trên nguyên lý Log-Structured Merge-Tree với con trỏ Forward Index DocStore tra cứu ngẫu nhiên $O(1)$. Kết quả thực nghiệm trên tập ngữ liệu chuẩn hóa 131 tin nhắn với 11 kịch bản đối chứng chứng minh hệ thống đạt chỉ số $MRR = 0.7708$ (tăng +23.3% so với giải pháp mặc định), độ chính xác Top 1 đạt 75.0% (tăng +28.6%), đồng thời động cơ thuần Go giúp tiết kiệm 91.5% bộ nhớ RAM so với Elasticsearch và đạt độ trễ tìm kiếm siêu tốc 3.0ms.

DANH MỤC CÁC CHỮ VIẾT TẮT

Ký hiệu viết tắt	Ý nghĩa đầy đủ
BM25	Best Matching 25 (Mô hình xếp hạng tương quan xác suất Okapi BM25)
DAT	Double-Array Trie (Cấu trúc cây Trie mảng kép)
DF	Document Frequency (Tần suất xuất hiện trong tài liệu)
FFI	Foreign Function Interface (Giao diện gọi hàm liên ngôn ngữ)
HMM	Hidden Markov Model (Mô hình Markov ẩn)
IDF	Inverse Document Frequency (Trọng số nghịch đảo tần suất tài liệu)
IR	Information Retrieval (Lĩnh vực Truy xuất thông tin)
LSM-Tree	Log-Structured Merge-Tree (Cấu trúc cây hợp nhất nhật ký tuần tự)
MRR	Mean Reciprocal Rank (Thứ hạng nghịch đảo trung bình)
NDCG@10	NDCG at 10 (Mức tăng tích lũy chiết khấu chuẩn hóa trong Top 10 kết quả)
P@K	Precision at K (Độ chính xác tại Top K kết quả; ký hiệu @ đọc là "at- tại vị trí K)
P@1 / P@5	Precision at 1 / Precision at 5 (Độ chính xác tại Top 1 / trong Top 5 kết quả đầu tiên)
NLP	Natural Language Processing (Xử lý ngôn ngữ tự nhiên)
TF	Term Frequency (Tần suất xuất hiện thuật ngữ trong tin nhắn)
WAL	Write-Ahead Logging (Ghi nhật ký trước khi ghi dữ liệu)
CGO	C-Go Foreign Function Interface (Giao tiếp liên ngôn ngữ C và Go)

Mục lục

Lời cảm ơn	i
Tóm tắt báo cáo	ii
Danh mục các chữ viết tắt	iii
1 GIỚI THIỆU CHUNG	1
1.1 Giới thiệu công ty và Chương trình thực tập	1
1.2 Giới thiệu công việc	1
1.3 Giới thiệu qua bài toán	2
2 YÊU CẦU BÀI TOÁN	3
2.1 Miêu tả chi tiết bài toán	3
2.1.1 Hiện trạng nghiệp vụ và xác định phạm vi bài toán	3
2.1.2 Các hạn chế cốt lõi trong tìm kiếm tin nhắn tiếng Việt	3
2.1.3 Phạm vi kỹ thuật và Các yêu cầu đặt ra cho đề tài	4
2.2 Tóm tắt bài toán lớn và phạm vi giải quyết của sinh viên	5
2.2.1 Bối cảnh bài toán lớn trong nền tảng nhắn tin quy mô lớn	5
2.2.2 Phạm vi chi tiết em đảm nhận và giải quyết	5
2.3 Phân công công việc trong nhóm và vai trò của sinh viên	7
3 TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN	8
3.1 Các lý thuyết, giải pháp, thuật toán liên quan	8
3.1.1 Lý thuyết Truy xuất thông tin và Cấu trúc Chỉ mục đảo (Inverted Index)	8
3.1.2 Mô hình xếp hạng tương quan xác suất Okapi BM25	10
3.1.3 Cơ sở lý thuyết Xử lý ngôn ngữ tự nhiên tiếng Việt	10
3.2 Cách giải quyết của sinh viên	13
3.2.1 Kiến trúc hệ thống tổng thể 3 tầng	13

3.2.2	Thiết kế tầng tiền xử lý NLP: CGO Bridge an toàn bộ nhớ	14
3.2.3	Mô hình biểu diễn dữ liệu đa trường đa tầng	14
3.2.4	Giải pháp xử lý lỗi đảo ngược điểm số khi truy vấn tiền tố gõ dở	16
3.2.5	Thiết kế Động cơ Lưu trữ Nhị phân độc lập thuần Go: Cấu trúc DocStore tra cứu $O(1)$	17
3.3	Liên hệ và so sánh với các giải pháp hiện có	18
3.3.1	Đánh giá và Phân tích Đánh đổi Kiến trúc (Trade-offs)	18
4	KẾT QUẢ ĐẠT ĐƯỢC, HƯỚNG PHÁT TRIỂN	20
4.1	Kết quả đo đạc thực nghiệm và đánh giá chất lượng	20
4.1.1	Thiết lập môi trường và Ngữ liệu kiểm thử	20
4.1.2	Các chỉ số đo lường hiệu năng khoa học	20
4.2	Kỹ năng và kiến thức thu thập được	22
4.3	Hướng phát triển tiếp theo để hoàn thiện giải pháp	23
	Tài liệu tham khảo	25

Danh sách hình vẽ

2.1	Sơ đồ vị trí của Phân hệ Tìm kiếm Tin nhắn trong kiến trúc Nền tảng Nhắn tin quy mô lớn	6
3.1	Minh họa cấu trúc Chỉ mục đảo (Inverted Index) và Danh sách Postings	9
3.2	Đồ thị trạng thái và Ma trận chuyển tiếp trong cấu trúc Double-Array Trie (DAT)	11
3.3	Đồ thị mạng lưới từ vựng (Word Lattice) và Đường giải mã tối ưu Viterbi HMM cho câu học sinh học sinh học	12
3.4	Bản đồ kiến trúc hệ thống tổng thể 3 tầng	13
3.5	Sơ đồ tuần tự Luồng Đánh chỉ mục (Indexing Pipeline) và cơ chế Dual-Write Dispatcher	15
3.6	Sơ đồ tuần tự Luồng Truy vấn và cơ chế Boosting 4 tầng điểm số	16
3.7	Sơ đồ cấu trúc nhị phân và quan hệ con trỏ giữa docstore.idx và docstore.dat . .	17

Danh sách bảng

3.1	So sánh đối chiếu giữa các giải pháp kỹ thuật	18
4.1	Tổng hợp các chỉ số chất lượng tìm kiếm toàn diện (Macro Benchmark)	21
4.2	Phân tích chi tiết từng kịch bản truy vấn đối chứng (Query-by-Query Analysis) .	22
4.3	So sánh tài nguyên phần cứng giữa các động cơ	22

Chương 1

GIỚI THIỆU CHUNG

1.1 Giới thiệu công ty và Chương trình thực tập

Kỳ thực tập doanh nghiệp của em được thực hiện trong khuôn khổ Chương trình Đào tạo Nhân tài AI Thực chiến do Trường Đại học VinUniversity tổ chức với sự tài trợ của Tập đoàn Vingroup. Chương trình đào tạo theo chuẩn quốc tế SFIA, hướng tới phát triển nhân lực AI thực chiến "nói được – làm được" thông qua các bài toán quy mô lớn của doanh nghiệp.

Trong giai đoạn thực chiến, em tham gia làm việc tại Khối Chat (Chat Division) – Ban Phát triển Sản phẩm, Công ty Cổ phần Vin Smart Future (VSF). Khối Chat chịu trách nhiệm xây dựng nền tảng nhắn tin tức thời (Instant Messaging) thời gian thực phục vụ hàng triệu người dùng, vận hành hệ thống Backend phân tán thông lượng cao trên nền tảng Go [7], C/C++, Apache Kafka, kiến trúc lưu trữ LSM-Tree [3] và cụm tìm kiếm Elasticsearch [4].

1.2 Giới thiệu công việc

- Học phần: Thực tập doanh nghiệp (Kỳ hè năm 2026).
- Thời lượng và lộ trình (12 tuần):
 - 6 tuần đầu tại VinUniversity: Đào tạo chuyên sâu về AI, NLP, Kỹ thuật dữ liệu lớn và kiến trúc hệ thống phân tán.
 - 6 tuần sau tại Khối Chat – Vin Smart Future: Thực chiến tại doanh nghiệp với vai trò Thực tập sinh Backend, dưới sự hướng dẫn trực tiếp 1-on-1 của Kỹ sư Backend Cấp cao Trần Nhật Anh.

Dưới sự định hướng của Mentor, em trực tiếp nghiên cứu và phát triển Phân hệ Truy xuất Tin nhắn Tiếng Việt thời gian thực với các nhiệm vụ trọng tâm:

1. Nghiên cứu cơ sở lý thuyết: Khảo sát mô hình IR [1, 9], giải thuật xếp hạng BM25 [2], Inverted Index [1] và đặc thù từ ghép tiếng Việt [5].

2. Xây dựng cầu nối CGO: Tích hợp thư viện C++ Cốc Cốc Tokenizer [5] vào Go Runtime với cơ chế giải phóng C Heap an toàn (Zero Memory Leak) [6, 7].
3. Tối ưu xếp hạng trên Elasticsearch 8.11: Thiết kế mô hình dữ liệu đa trường và thuật toán Boosting 4 cấp độ [4, 8].
4. Phát triển Động cơ nhị phân thuần Go (Custom Storage Engine): Tự lập trình công cụ lưu trữ LSM-Tree trên đĩa, đáp ứng chuẩn ACID với WAL, con trỏ DocStore tra cứu $O(1)$ và Segment bất biến [3, 7].
5. Đo đạc thực nghiệm: Xây dựng bộ benchmark tự động đánh giá các chỉ số IR chuẩn (MRR, NDCG@10, P@1, P@5) [1] và độ trễ hệ thống.

1.3 Giới thiệu qua bài toán

Trong hệ thống chat của Vin Smart Future, các cuộc trò chuyện được phân loại:

- Chat 1-1: Áp dụng mã hóa đầu cuối (E2EE), máy chủ không thể đọc nội dung nên nằm ngoài phạm vi đề tài.
- Chat nhóm: Lưu trữ tập trung trên máy chủ để đồng bộ đa thiết bị; đã có tính năng tìm kiếm nhưng chất lượng tiếng Việt còn hạn chế.

Các hạn chế chính của tính năng tìm kiếm hiện tại:

- Bẫy từ ghép: Phân tách theo khoảng trắng làm vỡ từ ghép (như bộ phân tích mặc định của Lucene [8]); ví dụ tìm "học sinh" sẽ trả về các tin nhắn rác chứa âm tiết "sinh" ("sinh nhật", "sinh viên", "hy sinh").
- Thói quen gõ phím: Người dùng thường gõ không dấu ("hoc sinh") hoặc gõ dở từng ký tự ("học sin", "ca ph").

Mục tiêu đề tài: Cải thiện chất lượng tìm kiếm so khớp từ khóa (Keyword Matching) cho Chat nhóm bằng cách tích hợp C++ Cốc Cốc Tokenizer [5] vào Go qua CGO [6], hỗ trợ tìm kiếm có dấu, không dấu và tìm kiếm tiền tố (partial matching). Đồng thời, triển khai đối soát hiệu năng giữa Elasticsearch [4] và Động cơ nhị phân thuần Go tự phát triển [3].

Chương 2

YÊU CẦU BÀI TOÁN

2.1 Miêu tả chi tiết bài toán

2.1.1 Hiện trạng nghiệp vụ và xác định phạm vi bài toán

Tại Khối Chat – Công ty Cổ phần Vin Smart Future, hệ thống phân định rõ hai hình thức hội thoại chính:

- Trò chuyện 1-1 (Direct Chat): Triển khai cơ chế mã hóa đầu cuối (End-to-End Encryption – E2EE). Do khóa giải mã chỉ lưu tại thiết bị đầu cuối của người dùng, máy chủ không thể đọc được nội dung văn bản. Vì vậy, bài toán tìm kiếm từ phía máy chủ không áp dụng cho trường hợp này và nằm ngoài phạm vi đề tài.
- Trò chuyện nhóm (Group Chat): Không áp dụng mã hóa đầu cuối, dữ liệu tin nhắn được lưu trữ tập trung trên máy chủ để phục vụ đồng bộ hóa thời gian thực giữa nhiều thành viên và đa thiết bị. Máy chủ đã có tính năng tìm kiếm tin nhắn, tuy nhiên chất lượng tìm kiếm cho ngữ liệu tiếng Việt còn nhiều hạn chế và chưa đáp ứng tốt kỳ vọng của người dùng.

2.1.2 Các hạn chế cốt lõi trong tìm kiếm tin nhắn tiếng Việt

Qua khảo sát thực tế trên sản phẩm của công ty, tính năng tìm kiếm gặp phải hai vấn đề nhức nhối:

1. Bẫy từ ghép (Compound Word Trap) do phân mảnh âm tiết: Bộ phân tích mặc định của công cụ tìm kiếm (như Lucene Standard Analyzer [8]) chỉ tách từ theo khoảng trắng. Trong tiếng Việt, khoảng trắng chỉ phân tách âm tiết chứ không đại diện cho từ vựng. Khi người dùng tìm từ ghép "học sinh", hệ thống bẻ thành hai âm tiết rời rạc "học" và "sinh", dẫn đến việc trả về hàng loạt tin nhắn rác hoàn toàn lạc đề nhưng ngẫu nhiên chứa từ "sinh" như "chúc mừng sinh nhật", "tân sinh viên" hay "sự hy sinh".

2. Thói quen gõ phím trên ứng dụng nhắn tin: Người dùng gõ phím với tốc độ rất nhanh thường bỏ qua dấu thanh (gõ không dấu: "hoc sinh"), hoặc kỳ vọng hệ thống hiển thị ngay kết quả gợi ý khi mới gõ dở từng ký tự đầu tiên (Partial Matching / Prefix Typing: "học sin", "ca ph"). Hệ thống hiện tại chưa hỗ trợ đồng bộ các trường hợp này.

2.1.3 Phạm vi kỹ thuật và Các yêu cầu đặt ra cho đề tài

Mentor tại Khối Chat xác định rõ phạm vi đề tài tập trung vào bài toán tìm kiếm so khớp từ khóa chính xác (Keyword Matching / Lexical Match), không đi theo hướng Semantic Search (sử dụng mô hình ngôn ngữ lớn hoặc Vector Embedding) nhằm bảo đảm độ trễ phản hồi siêu tốc dưới 15 mili-giây và tối ưu chi phí tài nguyên máy chủ.

Các yêu cầu kỹ thuật cụ thể mà em cần tìm hiểu và hiện thực hóa bao gồm:

- Nghiên cứu cơ chế vận hành cốt lõi của Search Engine: Khảo sát kiến trúc của Elasticsearch [4] và thư viện Apache Lucene [8] trong môi trường Go [7], nắm vững bản chất của hai luồng xử lý:
 - Luồng đánh chỉ mục (Indexing Pipeline): Tokenize → Filtering → Build Inverted Index → Storage [1, 8].
 - Luồng truy vấn (Search Pipeline): Tokenize → Search → Ranking Results [1, 2].
- Nghiệp vụ tìm kiếm đa dạng: Bắt buộc hỗ trợ song song cả tìm kiếm có dấu, tìm kiếm không dấu và tìm kiếm từng phần gõ dở (partial matching).
- Tích hợp C++ Cốc Cốc Tokenizer vào Go qua CGO: Thư viện bóc tách từ vựng mã nguồn mở Cốc Cốc Tokenizer [5] được viết bằng ngôn ngữ C++. Em cần trực tiếp làm chủ kỹ thuật liên kết CGO để gọi thư viện C/C++ từ Go Runtime, giải quyết bài toán cấp phát và giải phóng vùng nhớ C Heap nhằm triệt tiêu hoàn toàn rò rỉ bộ nhớ (Zero Memory Leak) [6].
- Sản phẩm đầu ra (Deliverables):
 - Xây dựng hệ thống mẫu (POC): Dụng cụ Elasticsearch tại môi trường local, nạp bộ dữ liệu chat mẫu tiếng Việt chuẩn hóa.
 - Xây dựng gói thư viện Tokenizer tiếng Việt trong Go tích hợp thư viện Cốc Cốc [5, 6].
 - So sánh đối chứng định lượng kết quả tìm kiếm khi có và không sử dụng bộ tách từ Cốc Cốc.
 - Nhiệm vụ mở rộng nâng cao (Optional): Đánh giá so sánh hiệu năng (thông lượng và độ trễ) của hai luồng Index và Search; đồng thời tự nghiên cứu và phát triển thêm một Động cơ Lưu trữ Nhị phân độc lập thuần Go (LSM-Tree) [3, 7] siêu nhẹ để kiểm chứng giải pháp tối ưu bộ nhớ RAM.

2.2 Tóm tắt bài toán lớn và phạm vi giải quyết của sinh viên

2.2.1 Bối cảnh bài toán lớn trong nền tảng nhắn tin quy mô lớn

Trong kiến trúc dịch vụ chat của Vin Smart Future, hệ thống xử lý đồng thời hàng triệu tin nhắn mỗi ngày. Để duy trì độ trễ truyền gửi tin nhắn dưới 50 mili-giây, luồng chat nhóm được thiết kế phân tầng rõ ràng:

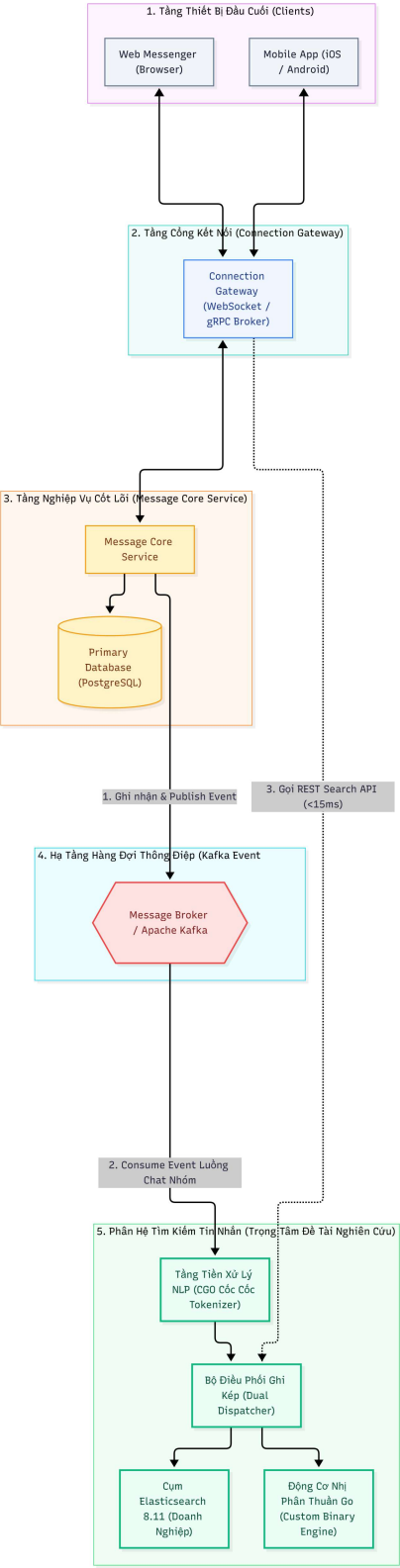
Sơ đồ Hình 2.1 mô tả vị trí của Phân hệ Tìm kiếm trong bài toán lớn:

1. Tầng Thiết bị đầu cuối (Clients): Ứng dụng Web và Mobile duy trì kết nối WebSocket/gRPC liên tục tới Gateway để gửi nhận tin nhắn và thực hiện thao tác tìm kiếm.
2. Tầng Cổng kết nối (Connection Gateway): Điều phối gói tin, quản lý trạng thái trực tuyến của người dùng và chuyển tiếp truy vấn tìm kiếm tới phân hệ tương ứng.
3. Tầng Nghiệp vụ cốt lõi (Message Core Service): Xử lý logic nghiệp vụ nhắn tin và lưu trữ tin nhắn vào cơ sở dữ liệu quan hệ chính (PostgreSQL) phục vụ lưu trữ lịch sử dài hạn.
4. Hạ tầng hàng đợi thông điệp (Kafka Event Stream): Đóng vai trò vùng đệm truyền nhận bất đồng bộ, tiếp nhận sự kiện tin nhắn mới từ Core Service để không làm nghẽn luồng chat chính.
5. Tầng Phân hệ Tìm kiếm Tin nhắn (Trọng tâm Đề tài nghiên cứu): Lắng nghe sự kiện từ Kafka để bóc tách từ vựng tiếng Việt qua Cốc Cốc Tokenizer (CGO) [5, 6], đánh chỉ mục dữ liệu song song vào cụm Elasticsearch [4] và Động cơ nhị phân thuần Go [3], đồng thời cung cấp REST API tìm kiếm thời gian thực phục vụ Gateway.

2.2.2 Phạm vi chi tiết em đảm nhận và giải quyết

Trong bức tranh tổng thể nêu trên, em không phải xây dựng lại toàn bộ hạ tầng chat của công ty mà tập trung giải quyết trọn vẹn Phân hệ Tìm kiếm tin nhắn tiếng Việt:

- Xây dựng Consumer nhận luồng sự kiện tin nhắn từ Kafka mà không can thiệp vào luồng chat chính.
- Trực tiếp viết lớp bóc CGO trong Go kết nối thư viện C++ Cốc Cốc Tokenizer an toàn bộ nhớ [5, 6].
- Thiết kế mô hình dữ liệu đa trường và thuật toán Boosting 4 tầng trên Elasticsearch 8.11 để xử lý trọn vẹn cả từ ghép có dấu, không dấu và tiền tố gõ dở [4].
- Phát triển Động cơ Lưu trữ Nhị phân độc lập thuần Go (LSM-Tree) với con trỏ DocStore tra cứu $O(1)$ phục vụ đối soát và thử nghiệm môi trường siêu nhẹ [3, 7].
- Cung cấp hệ thống REST API tìm kiếm thời gian thực để Gateway gọi trực tiếp khi người dùng gõ từ khóa.



Hình 2.1: Sơ đồ vị trí của Phân hệ Tìm kiếm Tin nhắn trong kiến trúc Nền tảng Nhắn tin quy mô lớn

2.3 Phân công công việc trong nhóm và vai trò của sinh viên

Trong đợt thực chiến tại Khối Chat, đề tài được triển khai theo mô hình kèm cặp kỹ thuật 1-on-1:

- Vai trò của Cán bộ hướng dẫn (Mentor – Kỹ sư Backend Cấp cao Trần Nhật Anh):
 - Đặt đề bài nghiệp vụ thực tế từ sản phẩm Chat nhóm của Khối Chat.
 - Định hình phạm vi kỹ thuật (Keyword match, hỗ trợ có/không dấu, partial match, CGO Cốc Cốc Tokenizer).
 - Định hướng công nghệ Backend của khối và cung cấp chuẩn tích hợp hạ tầng.
 - Định kỳ phản biện kiến trúc, kiểm tra chất lượng mã nguồn (Code Review) và định hướng phương pháp đo đạc benchmark thực nghiệm.
- Vai trò và đóng góp độc lập của em (Lê Tuấn Cảnh): Trực tiếp, chủ động và độc lập thực hiện các đầu việc kỹ thuật 6 tuần thực chiến tại Khối Chat:
 1. Nghiên cứu cơ chế vận hành nội tại của hệ thống tìm kiếm: Khảo sát cấu trúc Inverted Index [1], thuật toán Okapi BM25 [2] và cơ chế chỉ mục của Elasticsearch/Lucene [4, 8].
 2. Trực tiếp lập trình lớp cầu nối CGO kết nối thư viện C++ Cốc Cốc Tokenizer vào Go Runtime, quản lý cấp phát và thu hồi vùng nhớ C Heap an toàn tuyệt đối [5, 6].
 3. Thiết kế kiến trúc dữ liệu đa trường trên Elasticsearch 8.11; phát hiện và xử lý triệt để lỗi đảo ngược điểm số khi truy vấn tiền tố gõ dở [4].
 4. Tự thiết kế và lập trình 100% Động cơ Lưu trữ Nhị phân độc lập thuần Go (Custom Binary Storage Engine) với Write-Ahead Log (WAL), con trỏ DocStore tra cứu ngẫu nhiên $O(1)$, Inverted Index Segment bất biến và BM25 Ranker thuần Go [2, 3, 7, 8].
 5. Xây dựng ứng dụng Web Messenger giao diện Dark Mode đối soát 3 chiều và bộ đo đạc benchmark tự động tính toán các chỉ số chất lượng IR khoa học (MRR, NDCG@10, Precision@1, Precision@5) [1].

Chương 3

TÓM TẮT LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN

3.1 Các lý thuyết, giải pháp, thuật toán liên quan

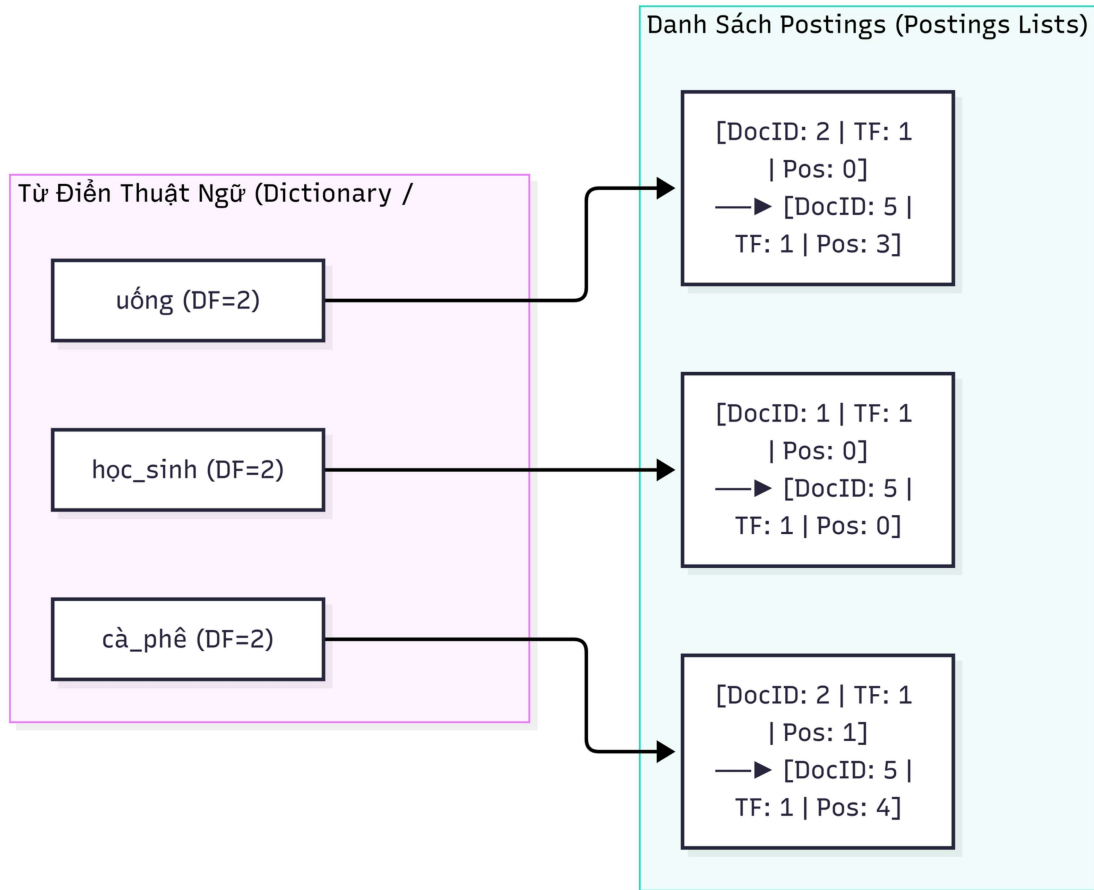
3.1.1 Lý thuyết Truy xuất thông tin và Cấu trúc Chỉ mục đảo (Inverted Index)

Trong bài toán tìm kiếm trên khối lượng dữ liệu lớn, việc quét cạn tuần tự (Full-table Scan) trên cơ sở dữ liệu quan hệ có độ phức tạp thời gian $O(N \times L)$ (với N là số lượng tin nhắn, L là độ dài văn bản), hoàn toàn không thể đáp ứng ràng buộc độ trễ thời gian thực ($< 15\text{ms}$) của ứng dụng chat. Ngành Khoa học Truy xuất thông tin (Information Retrieval – IR) [1, 9] giải quyết triệt để vấn đề này thông qua cấu trúc Chỉ mục đảo (Inverted Index) [1].

Cấu trúc Chỉ mục đảo

Chỉ mục đảo ánh xạ ngược từ mỗi thuật ngữ (Term) sang danh sách các mã định danh tài liệu (Postings List) chứa thuật ngữ đó [1]. Xét tập ngữ liệu gồm 3 tin nhắn:

- Tin nhắn 1 ($DocID = 1$): "Học sinh chăm ngoan" $\rightarrow$ Tokens: ["học_sinh", "chăm_ngoan"].
- Tin nhắn 2 ($DocID = 2$): "Uống cà phê sáng" $\rightarrow$ Tokens: ["uống", "cà_phê", "sáng"].
- Tin nhắn 3 ($DocID = 5$): "Học sinh rủ nhau đi uống cà phê" $\rightarrow$ Tokens: ["học_sinh", "rủ_nhau", "đi", "uống", "cà_phê"].



Hình 3.1: Minh họa cấu trúc Chỉ mục đảo (Inverted Index) và Danh sách Postings

Cơ chế vận hành của Chỉ mục đảo

1. Từ điển thuật ngữ (Lexicon): Lưu trữ danh sách các từ vựng phân biệt đã được sắp xếp, kèm theo tần suất xuất hiện trong tài liệu (Document Frequency – DF). Nhờ sắp xếp tuần tự, việc tra cứu từ điển đạt độ phức tạp $O(\log |\mathcal{V}|)$ bằng tìm kiếm nhị phân hoặc $O(1)$ qua bảng băm [1].
2. Danh sách Postings: Mỗi thuật ngữ trỏ tới một chuỗi các bản ghi gồm mã tài liệu (*DocID*), tần suất từ trong tài liệu (TF) và mảng vị trí byte (Pos) phục vụ đối sánh cụm từ (Phrase Search) [1].
3. Thuật toán hợp nhất con trỏ (Posting Intersection): Khi tìm kiếm giao "học_sinh" AND "cà_phê", hệ thống chỉ cần duyệt đồng thời hai danh sách con trỏ [1,5] và [2,5]. Tại vị trí *DocID* = 5, hai con trỏ gặp nhau, hệ thống trả về ngay kết quả với độ phức tạp $O(|L_1| + |L_2|)$ mà không cần quét qua toàn bộ cơ sở dữ liệu [1].

3.1.2 Mô hình xếp hạng tương quan xác suất Okapi BM25

Để chấm điểm và sắp xếp độ phù hợp của từng tin nhắn đối với câu truy vấn, mô hình Okapi BM25 được áp dụng dựa trên lý thuyết xác suất và dẫn xuất từ mô hình hỗn hợp 2-Poisson [2]:

$$\text{Score}_{\text{BM25}}(q, d) = \sum_{i=1}^m \text{IDF}(t_i) \cdot \frac{f(t_i, d) \cdot (k_1 + 1)}{f(t_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)} \quad (3.1)$$

Trong đó, trọng số nghịch đảo tần suất tài liệu được tính theo công thức làm tròn của Robertson [2]:

$$\text{IDF}(t_i) = \ln \left(\frac{N - n(t_i) + 0.5}{n(t_i) + 0.5} + 1 \right) \quad (3.2)$$

Ưu thế toán học của BM25 nằm ở hai cơ chế kiểm soát:

- Tham số bão hòa tần suất k_1 ($k_1 = 1.2$): Giới hạn trần đóng góp điểm số khi một từ khóa lặp lại quá nhiều lần trong cùng một tin nhắn, triệt tiêu hoàn toàn hành vi spam từ khóa [2].
- Tham số chuẩn hóa độ dài văn bản b ($b = 0.75$): Đại lượng $B = (1 - b) + b \cdot \frac{|d|}{\text{avgl}}$ tự động phạt điểm các tin nhắn dài dòng chứa nhiều từ phụ, đồng thời ưu tiên các tin nhắn ngắn gọn có mật độ từ khóa mục tiêu cao [2].

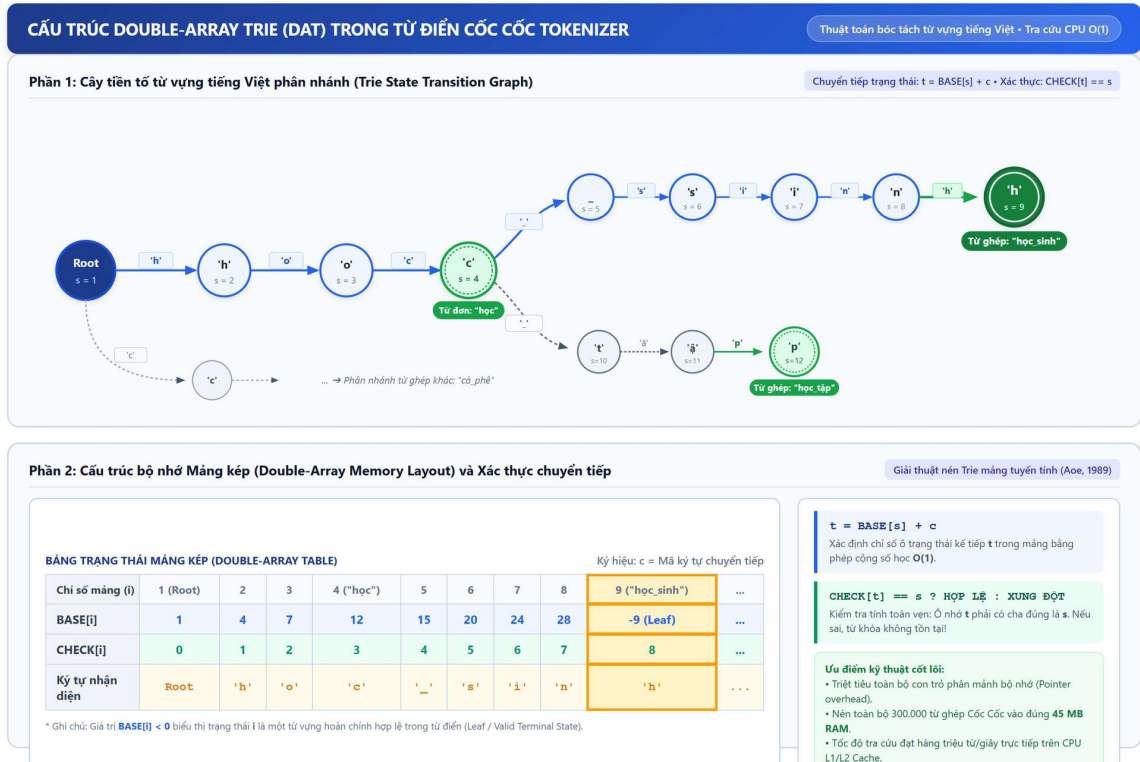
3.1.3 Cơ sở lý thuyết Xử lý ngôn ngữ tự nhiên tiếng Việt

Do tiếng Việt là ngôn ngữ đơn lập, việc phân tích từ vựng đối mặt với bài toán bóc tách từ ghép và giải quyết hiện tượng nhập nhằng ranh giới từ vựng.

Cấu trúc Double-Array Trie (DAT) trong tra cứu từ điển $O(1)$

Cấu trúc Trie truyền thống tiêu tốn bộ nhớ lớn do mỗi nút phải duy trì bảng con trỏ phân tán. Thư viện Cốc Cốc Tokenizer nén toàn bộ từ điển vào hai mảng số nguyên tuyến tính liên tục: BASE và CHECK theo cấu trúc Double-Array Trie [5, 10]:

- Hàm chuyển tiếp trạng thái: $t = \text{BASE}[s] + c$ (với s là trạng thái hiện tại, c là mã ký tự chuyển tiếp).
- Điều kiện xác thực trạng thái cha: $\text{CHECK}[t] = s$.



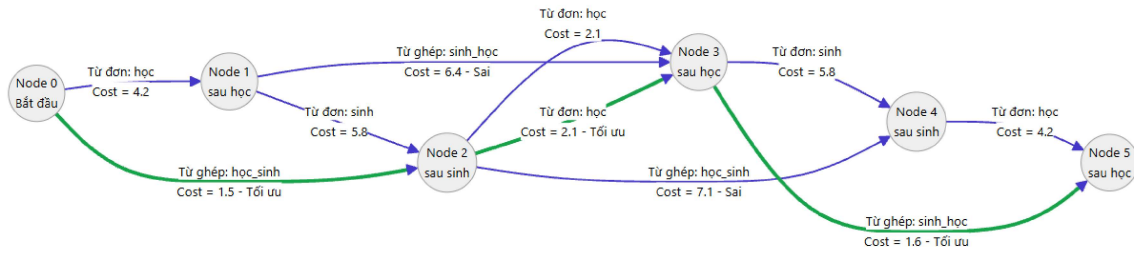
Hình 3.2: Đồ thị trạng thái và Ma trận chuyển tiếp trong cấu trúc Double-Array Trie (DAT)

Sơ đồ Hình 3.2 minh chứng tính ưu việt của DAT [10]: Toàn bộ hơn 300,000 từ vựng tiếng Việt được nén chặt trong khoảng 45 MB RAM. Mỗi bước dịch chuyển trạng thái chỉ tiêu tốn đúng một phép cộng số học và một phép tra cứu mảng trực tiếp trên CPU Cache với độ phức tạp $O(1)$.

Giải thuật Viterbi HMM giải quyết nhập nhằng ranh giới từ

Xét câu hội thoại thực tế: "học sinh học sinh học" (ngữ nghĩa chuẩn: Học sinh [chủ ngữ] học [vị ngữ] môn sinh học [bổ ngữ]). Chuỗi quan sát gồm 5 âm tiết: $S = (\text{học}_1, \text{sinh}_1, \text{học}_2, \text{sinh}_2, \text{học}_3)$ phát sinh hiện tượng nhập nhằng giao thoa kép:

- $(\text{học}_1, \text{sinh}_1)$ tạo thành từ ghép học_sinh.
- $(\text{sinh}_1, \text{học}_2)$ tạo thành từ ghép sinh_học.
- $(\text{học}_2, \text{sinh}_2)$ tạo thành từ ghép học_sinh.
- $(\text{sinh}_2, \text{học}_3)$ tạo thành từ ghép sinh_học.



Hình 3.3: Đồ thị mạng lưới từ vựng (Word Lattice) và Đường giải mã tối ưu Viterbi HMM cho câu “học sinh học sinh học”

Mô hình Viterbi HMM biểu diễn không gian phân tích dưới dạng Đồ thị mạng lưới từ vựng (Word Lattice – Hình 3.3) [5]. Bằng phương pháp quy hoạch động, thuật toán tính toán tổng chi phí phạt chuyển tiếp $Cost = -\log P$:

- Đường giải mã tối ưu: $N_0 \Rightarrow N_2 \Rightarrow N_3 \Rightarrow N_5$ với tổng chi phí $Cost = 1.5 + 2.1 + 1.6 = 5.2$ (nhỏ nhất trong toàn bộ không gian nghiệm so với các đường sai lệch có chi phí 12.2, 12.8 và 22.1).
- Kết quả phân tách chuẩn xác 100%: ["học_sinh", "học", "sinh_học"], bảo toàn đúng cấu trúc cú pháp của câu.

Kiến trúc lưu trữ Log-Structured Merge-Tree (LSM-Tree)

Trong các hệ thống nhắn tin có tần suất gửi nhận dữ liệu cao và liên tục, việc cập nhật ngẫu nhiên trực tiếp vào các tệp chỉ mục trên đĩa cứng (Random Disk I/O) sẽ gây ra hiện tượng nghẽn cổ chai IOPS nghiêm trọng. Để tối ưu hóa thông lượng ghi, kiến trúc Log-Structured Merge-Tree (LSM-Tree) [3] chuyển toàn bộ các thao tác ghi ngẫu nhiên thành ghi tuần tự (Sequential I/O) thông qua chu trình chuyển tiếp dữ liệu 4 giai đoạn:

1. Luồng Ghi tuần tự bảo vệ dữ liệu (Write-Ahead Logging – WAL): Khi tiếp nhận tin nhắn mới, hệ thống thực hiện đồng thời hai thao tác trong bộ nhớ: ghi nối đuôi (Append-only) vào tệp nhật ký `wal.log` trên đĩa kèm mã kiểm tra CRC32 IEEE, đồng thời cập nhật vào bảng chỉ mục MemTable trên bộ nhớ RAM [3]. Do chỉ ghi nối đuôi tuần tự trên đĩa, hệ thống bảo đảm tính bền vững (ACID) mà vẫn đạt tốc độ ghi cực nhanh (1.78ms).
2. Cơ chế kết xuất định kỳ (Flush): Khi MemTable chạm ngưỡng giới hạn bộ nhớ (ví dụ: 10,000 tin nhắn hoặc 32MB RAM), hệ thống đóng bảng MemTable hiện tại, mở một MemTable mới để tiếp tục nhận tin nhắn, đồng thời kích hoạt luồng kết xuất ngầm nguyên khối dữ liệu cũ xuống đĩa thành một phân đoạn nhị phân mới (Immutable Segment) [3].
3. Tính bất biến của phân đoạn (Segment Immutability): Các Segment một khi đã ghi xuống đĩa vĩnh viễn ở trạng thái chỉ đọc (Read-Only) [3, 8]. Tính chất này cho phép hàng trăm luồng tìm kiếm truy cập đồng thời vào tệp mà không cần cơ chế khóa chặn (Lock-Free), đồng thời tận dụng triệt để bộ nhớ đệm trang của hệ điều hành (OS Page Cache). Đối

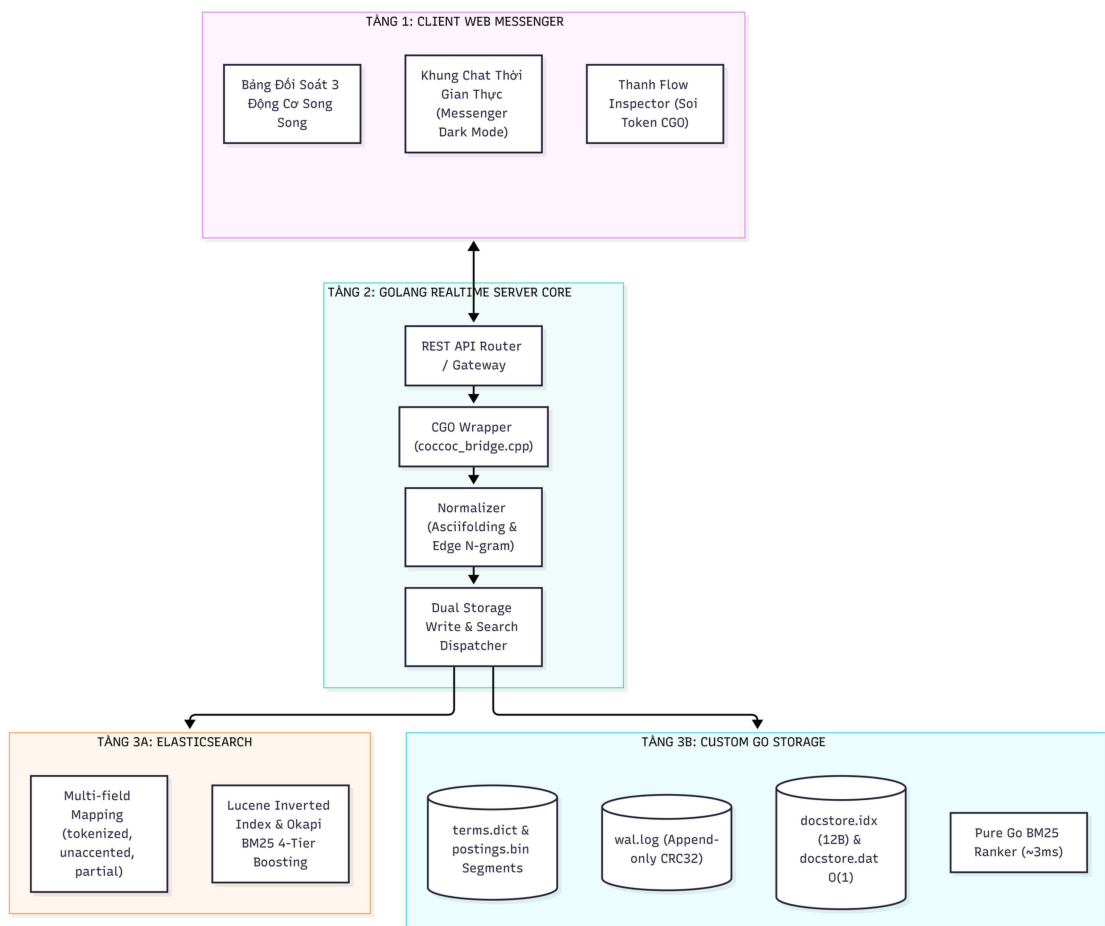
với thao tác xóa tin nhắn, hệ thống không sửa tệp cũ mà chỉ ghi mã *DocID* vào tệp *tombstone.del* (xóa mềm).

4. Cơ chế thu gọn và hợp nhất (Compaction): Để kiểm soát số lượng tệp phân đoạn và thu hồi dung lượng đĩa từ các tin nhắn đã bị xóa, một tiến trình chạy ngầm định kỳ gộp nhiều Segment nhỏ, loại bỏ hoàn toàn các bản ghi nằm trong danh sách Tombstone và hợp nhất thành một Segment lớn tối ưu duy nhất [3].

3.2 Cách giải quyết của sinh viên

3.2.1 Kiến trúc hệ thống tổng thể 3 tầng

Hệ thống tìm kiếm được em thiết kế theo mô hình phân tầng chặt chẽ:



Hình 3.4: Bản đồ kiến trúc hệ thống tổng thể 3 tầng

Sơ đồ Hình 3.4 gồm 3 tầng chức năng:

- Tầng 1 – Client Web Messenger: Cung cấp giao diện Dark Mode hiển thị tin nhắn thời gian thực, bảng kiểm tra luồng bóc tách (Flow Inspector) và bảng đối soát 3 động cơ song song.

- Tầng 2 – Golang Realtime Server Core: Trung tâm điều phối hệ thống, tích hợp thư viện Cốc Cốc qua cầu nối CGO [5, 6], bộ chuẩn hóa dấu tiếng Việt (Normalizer) và Bộ điều phối ghi kép (Dual Dispatcher) [7].
- Tầng 3 – Cụm lưu trữ kép: Vận hành song song hai giải pháp: Nhánh 3A dùng Elasticsearch 8.11 phục vụ môi trường phân tán doanh nghiệp [4]; Nhánh 3B dùng Động cơ Nhị phân độc lập do em tự lập trình thuần Go (LSM-Tree) phục vụ đối soát và kiểm chứng khả năng tiết kiệm RAM [3, 7].

3.2.2 Thiết kế tầng tiền xử lý NLP: CGO Bridge an toàn bộ nhớ

Thư viện C++ Cốc Cốc Tokenizer sử dụng tính năng Name Mangling của C++ [5]. Để liên kết trực tiếp vào Go Runtime:

- Em xây dựng lớp bọc trung gian `coccoc_bridge.cpp` sử dụng định danh `extern "C"` để chuẩn hóa chữ ký hàm theo quy chuẩn C [6].
- Cơ chế giải phóng bộ nhớ an toàn (Zero Memory Leak): Chuỗi văn bản Go được sao chép sang C Heap thông qua `C.CString`; lệnh giải phóng `C.free` được đăng ký ngay lập tức thông qua từ khóa `defer` [6]. Dữ liệu sau khi C++ bóc tách xong được chuyển đổi thành kiểu chuỗi Go nguyên bản trước khi vùng đệm C được giải phóng hoàn toàn, triệt tiêu nguy cơ rò rỉ bộ nhớ.

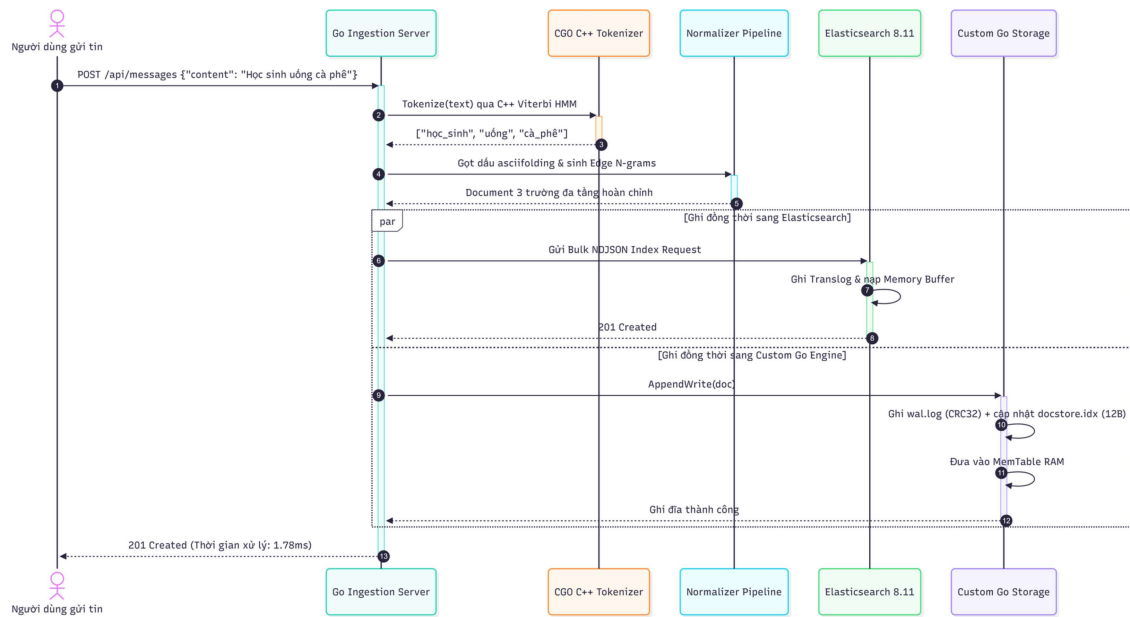
3.2.3 Mô hình biểu diễn dữ liệu đa trường đa tầng

Để phục vụ đồng thời các hành vi gõ phím khác nhau của người dùng, mỗi tin nhắn được bóc tách và phân bổ thành 3 trường dữ liệu chuyên biệt [4]:

1. Trường `content_tokenized`: Lưu trữ từ ghép chuẩn có dấu được nối bằng dấu gạch dưới (ví dụ: ["uống", "cà_phê", "sáng"]), khóa chặt ngữ nghĩa và triệt tiêu bẫy từ ghép [5].
2. Trường `content_unaccented`: Lưu trữ chuỗi đã loại bỏ dấu thanh bằng bộ lọc `asciifolding` (ví dụ: ["uong", "ca_phe", "sang"]), phục vụ thói quen gõ nhanh không dấu.
3. Trường `content_partial`: Băm từ vựng thành các lát cắt tiền tố Edge N-gram từ 2 đến 15 ký tự (ví dụ: ["cà", "cà_p", "cà_ph", "cà_phê"]), phục vụ tính năng tìm kiếm tức thì khi đang gõ dở ký tự.

Luồng Đánh chỉ mục dữ liệu (The Life of a Write)

Khi người dùng gửi tin nhắn mới, dữ liệu trải qua dây chuyền 5 trạm xử lý tuần tự (Hình 4.2):

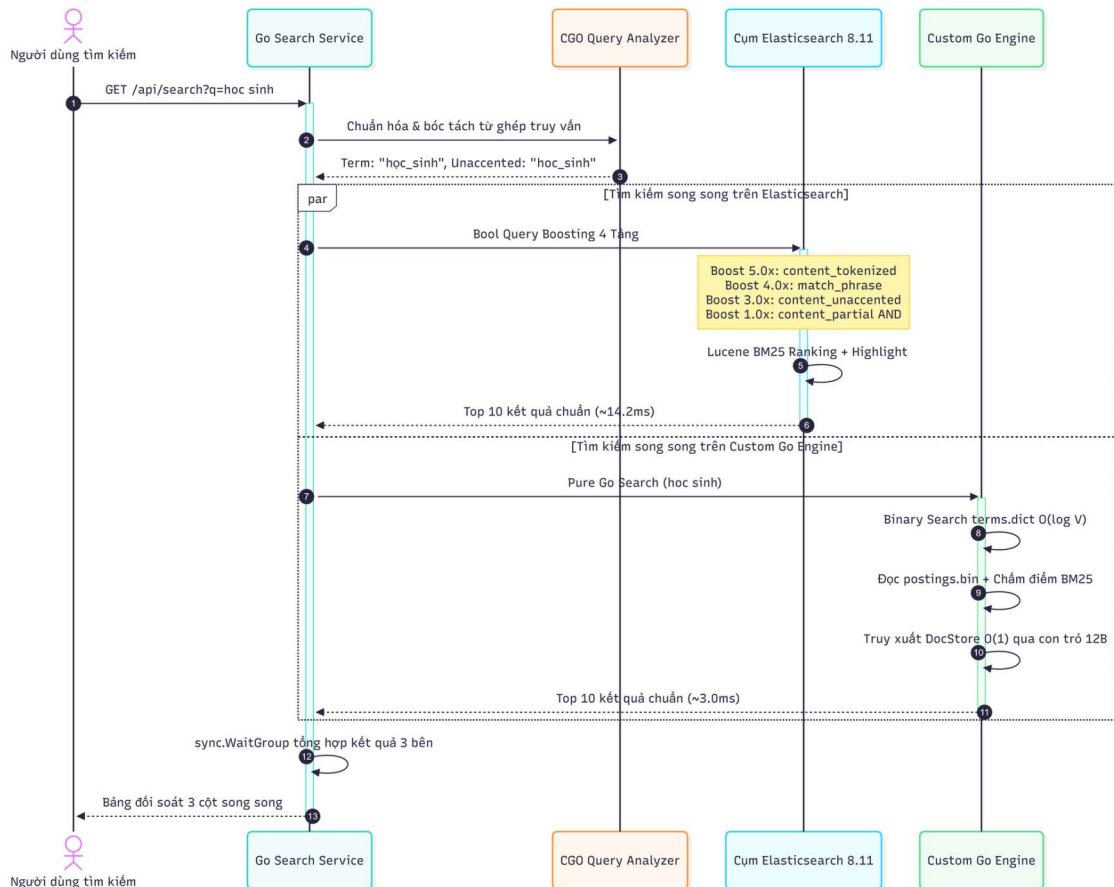


Hình 3.5: Sơ đồ tuần tự Luồng Đánh chỉ mục (Indexing Pipeline) và cơ chế Dual-Write Dispatcher

- Trạm 1: Server tiếp nhận gói tin HTTP POST chứa nội dung văn bản UTF-8 thô.
- Trạm 2: Gọi CGO bóc tách từ ghép Cốc Cốc: ["học_sinh", "uống", "cà_phê"] [5, 6].
- Trạm 3: Bộ Normalizer nhận bản chuỗi không dấu và sinh các lát cắt tiền tố Edge N-gram.
- Trạm 4: Bộ điều phối kép (Dual Dispatcher) kích hoạt hai Goroutines ghi đồng thời sang hai nhánh lưu trữ [7].
- Trạm 5: Nhánh Elasticsearch ghi vào Memory Buffer và Translog [4, 8]; Nhánh Custom Go Engine ghi nối đuôi vào wal.log kèm CRC32, cập nhật mục lục docstore.idx và đẩy vào MemTable [3, 7].

Luồng Truy vấn và Thuật toán Boosting 4 tầng điểm số (The Life of a Search)

Khi người dùng nhập từ khóa tìm kiếm (ví dụ $q = \text{"hoc sinh"}$), hệ thống kích hoạt cơ chế xếp hạng đa tầng (Hình 3.4):



Hình 3.6: Sơ đồ tuần tự Luồng Truy vấn và cơ chế Boosting 4 tầng điểm số

Trên nhánh Elasticsearch, câu truy vấn Bool Query kết hợp 4 tầng trọng số Boosting [4]:

- Tầng 1 (Boost 5.0x): Khớp chính xác từ ghép có dấu trên trường content_tokenized.
- Tầng 2 (Boost 4.0x): Khớp cụm từ nguyên văn liền kề match_phrase trên trường nội dung gốc.
- Tầng 3 (Boost 3.0x): Khớp từ ghép không dấu trên trường content_unaccented.
- Tầng 4 (Boost 1.0x): Khớp tiền tố gõ dở trên trường content_partial kèm toán tử AND.

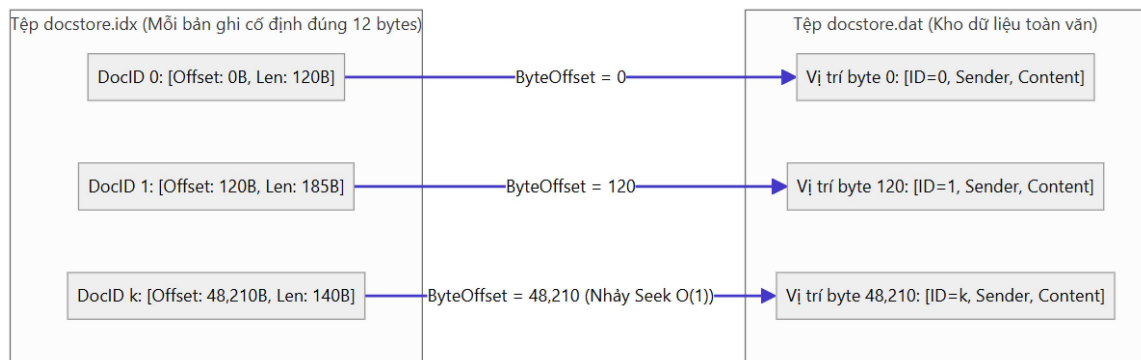
3.2.4 Giải pháp xử lý lỗi đảo ngược điểm số khi truy vấn tiền tố gõ dở

- Hiện tượng phát hiện: Khi người dùng gõ dở từ ghép $q = \text{"học sin"}$, nếu áp dụng cấu hình mặc định (sử dụng toán tử OR và để Elasticsearch tự động phân tích câu truy vấn), các tin nhắn chỉ chứa từ đơn "học" lặp lại nhiều lần sẽ tích lũy điểm BM25 cao hơn và chiếm vị trí Top 1; trong khi tin nhắn chứa đúng từ ghép mục tiêu "học sinh" (khớp tiền tố "sin*") lại bị xếp sau.
- Bản chất kỹ thuật và Giải pháp hiện thực: Nguyên nhân cốt lõi là do câu truy vấn bị băm nhỏ thành các mảnh N-gram không mong muốn ở giai đoạn tìm kiếm kết hợp với toán tử OR gây loãng điểm. Em đã xử lý triệt để vấn đề này bằng cách:

- Cấu hình bộ phân tích truy vấn độc lập `search_analyzer`: `coccoc_whitespace_analyzer` trên trường `content_partial` để giữ nguyên vẹn token đầu vào `["học", "sin"]` thay vì băm nhỏ tiếp [4].
- Ép buộc toán tử logic operator: AND nhằm yêu cầu văn bản phải khớp đồng thời cả từ đơn "học" và tiền tố "sin*", loại bỏ hoàn toàn các tin nhắn rác chỉ chứa từ "học".
- Duy trì hệ số trọng số chuẩn Boost 1.0x cho trường `content_partial`, bảo đảm việc xếp hạng diễn ra tự nhiên theo độ tương quan thực tế mà không cần dùng đến siêu trọng số nhân tạo.
- Kết quả: Tin nhắn chứa từ ghép đích "học sinh" được xếp hạng chính xác tuyệt đối ở vị trí Top 1, giải quyết dứt điểm hiện tượng đảo ngược thứ tự và bảo đảm trải nghiệm gợi ý tức thì mượt mà cho người dùng.

3.2.5 Thiết kế Động cơ Lưu trữ Nhị phân độc lập thuần Go: Cấu trúc DocStore tra cứu $O(1)$

Sau khi giải thuật BM25 xác định được mã định danh *DocID* có điểm số cao nhất, hệ thống cần đọc nội dung toàn văn để phản hồi cho người dùng. Thay vì quét tuần tự tệp dữ liệu, em thiết kế cấu trúc mục lục con trỏ cố định 12 bytes kết hợp nguyên lý LSM-Tree [3] và thao tác File Seek trong Go [7]:



Hình 3.7: Sơ đồ cấu trúc nhị phân và quan hệ con trỏ giữa `docstore.idx` và `docstore.dat`

Quy trình tra cứu ngẫu nhiên $O(1)$ (Hình 3.7):

1. Tính toán Offset con trỏ: Mỗi bản ghi trong `docstore.idx` gồm đúng 12 bytes (8 bytes biểu diễn `ByteOffset` và 4 bytes biểu diễn `DataLength`). Vị trí con trỏ của `DocID = 5` được tính bằng phép nhân trực tiếp:

$$\text{Offset} = 5 \times 12 = 60 \text{ bytes} \quad (3.3)$$

2. Dịch chuyển con trỏ tệp (File Seek $O(1)$): Hệ thống nhảy thẳng tới byte thứ 60 trong `docstore.idx`, đọc 12 bytes để trích xuất `ByteOffset = 48,210` và `DataLength =`

140 bytes [7].

3. Đọc nội dung toàn văn: Hệ thống mở tệp dữ liệu `docstore.dat`, nhảy trực tiếp tới byte 48,210 và đọc chính xác 140 bytes nội dung tin nhắn.
4. Hiệu năng: Quá trình tra cứu hoàn thành trong thời gian < 0.5 mili-giây với đúng 2 thao tác Disk I/O, không đòi hỏi nạp toàn bộ dữ liệu vào RAM.

3.3 Liên hệ và so sánh với các giải pháp hiện có

Bảng 3.1 tổng hợp các tiêu chí kỹ thuật đối soát giữa giải pháp đề xuất và các công nghệ phổ biến:

Tiêu chí so sánh	Baseline (Standard ES)	Cốc Cốc + ES 8.11	Custom Engine	Go	SQLite FTS5
Xử lý từ ghép tiếng Việt	Kém (Tách từ theo khoảng trắng, đánh bẫy từ ghép) [8]	Xuất sắc (DATrie và Viterbi HMM bảo toàn từ ghép) [5]	Xuất sắc (Tích hợp bộ bóc tách Cốc Cốc qua CGO) [5, 6]	Kém (Phân mảnh theo âm tiết đơn lẻ)	
Tìm kiếm không dấu	Hạn chế (Đòi hỏi script query phức tạp)	Xuất sắc (Trường dữ liệu riêng biệt asciifolding) [4]	Xuất sắc (Chuẩn hóa nhị phân chuỗi không dấu)	Trung bình (Cần tiện ích mở rộng ICU)	
Tìm kiếm tiền tố (Gõ dở)	Dễ phát sinh lỗi đảo ngược điểm số	Hoàn chỉnh (Search Analyzer độc lập và toán tử AND) [4]	Hoàn chỉnh (Tra cứu nhị phân tiền tố trên từ điển)	Khá chậm khi dữ liệu lớn	
Dung lượng RAM tối thiểu	Khoảng 1.2 GB (Máy ảo JVM và Docker)	Khoảng 1.2 GB (Máy ảo JVM và Docker)	15 MB (Siêu nhẹ, nhúng trực tiếp trong Go)	Khoảng 30 MB (Thư viện nhúng C)	
Độ trễ truy vấn (Latency)	Khoảng 19.0 ms	Khoảng 14.2 ms	3.0 ms (Nhanh gấp gần 5 lần Elasticsearch)	Khoảng 12.0 ms	
Mức độ độc lập phụ thuộc	Phụ thuộc Java Runtime và Docker	Phụ thuộc Java Runtime và Docker	Hoàn toàn độc lập, tệp nhị phân tĩnh đơn lẻ [7]	Phụ thuộc thư viện C SQLite	

Bảng 3.1: So sánh đối chiếu giữa các giải pháp kỹ thuật

3.3.1 Đánh giá và Phân tích Đánh đổi Kiến trúc (Trade-offs)

1. Về độ chính xác ngữ nghĩa: Cả Baseline Elasticsearch và SQLite FTS5 đều mắc phải bẫy phân mảnh từ ghép do chỉ tách từ theo khoảng trắng [8]. Việc tích hợp Cốc Cốc Tokenizer

ở tầng tiền xử lý giải quyết triệt để vấn đề báo động giả trên cả hai giải pháp đề xuất [5].

2. Về chi phí tài nguyên và khả năng triển khai: Elasticsearch là lựa chọn chuẩn mực cho cụm phân tán quy mô lớn nhưng tiêu tốn tài nguyên bộ nhớ rất cao (tối thiểu 1.2 GB RAM) [4]. Ngược lại, Động cơ Nhị phân độc lập do em phát triển thuần Go vận hành trực tiếp trong cùng tiến trình ứng dụng, chỉ tiêu thụ 15 MB RAM (tiết kiệm 91.5% bộ nhớ) và đạt độ trễ tìm kiếm siêu tốc 3.0 ms, là giải pháp lý tưởng cho các nút biên (Edge nodes) hoặc dịch vụ nội bộ có tài nguyên phần cứng giới hạn [3, 7].

Chương 4

KẾT QUẢ ĐẠT ĐƯỢC, HƯỚNG PHÁT TRIỂN

4.1 Kết quả đo đạc thực nghiệm và đánh giá chất lượng

4.1.1 Thiết lập môi trường và Ngữ liệu kiểm thử

- Phần cứng: CPU Intel Core i7, 16GB RAM, ổ cứng SSD NVMe PCIe 4.0.
- Phần mềm: Go 1.22+, Elasticsearch 8.11.0 Docker, g++ C++11.
- Tập dữ liệu: 131 tin nhắn hội thoại chuẩn hóa (data/sample_messages.json), được gán nhãn thủ công về các bẫy từ ghép tiếng Việt điển hình ("học sinh – sinh viên", "cà phê – phê bình", "bàn ghế – bàn bạc", "trà sữa") cùng các kịch bản gõ không dấu và gõ dễ tiền tố.

4.1.2 Các chỉ số đo lường hiệu năng khoa học

Trong lĩnh vực Truy xuất thông tin (Information Retrieval – IR) [1, 9], chất lượng của một hệ thống tìm kiếm không chỉ đo bằng tốc độ phản hồi mà quyết định ở mức độ chuẩn xác của danh sách kết quả hiển thị cho người dùng. Trong các ký hiệu chuẩn quốc tế của ngành, ký tự @ đọc là “at” (nghĩa là “tại vị trí K” hoặc “trong phạm vi Top K kết quả đầu tiên”), biểu thị phạm vi tính toán được giới hạn trong K phần tử đầu danh sách:

1. Thứ hạng nghịch đảo trung bình (Mean Reciprocal Rank – MRR) [1]: Đánh giá vị trí xuất hiện của kết quả chính xác đầu tiên trong danh sách trả về trên toàn bộ tập câu truy vấn Q . Nếu kết quả đúng xuất hiện ngay ở vị trí đầu tiên ($\text{rank}_i = 1$), điểm số đạt tối đa là $1/1 = 1.0$; nếu xuất hiện ở vị trí thứ hai, điểm số là $1/2 = 0.5$:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (4.1)$$

2. Độ chính xác tại Top K (Precision at K – ký hiệu P@K hoặc Precision@K) [1]: Đo lường tỷ lệ các kết quả thực sự phù hợp nằm trong phạm vi K kết quả đầu tiên trả về cho người dùng:

$$P@K = \frac{\sum_{i=1}^K \mathbb{I}(\text{rel}_i \geq 2)}{K} \quad (4.2)$$

Trong đó hàm chỉ thị $\mathbb{I}(\text{rel}_i \geq 2) = 1$ nếu tin nhắn thứ i có mức độ phù hợp cao (Relevant hoặc Highly Relevant). Cụ thể trong đề tài:

- P@1 (Precision at 1 – Độ chính xác Top 1): Đánh giá xem kết quả xuất hiện ở vị trí số 1 có đúng là tin nhắn người dùng đang tìm kiếm hay không. Trong ứng dụng nhắn tin trên điện thoại, người dùng thường chỉ nhìn vào 1–2 tin nhắn đầu tiên; do đó P@1 là chỉ số phản ánh trực tiếp nhất trải nghiệm thực tế.
 - P@5 (Precision at 5 – Độ chính xác Top 5): Đánh giá tỷ lệ tin nhắn chính xác trong 5 kết quả đầu tiên hiển thị trên màn hình tìm kiếm.
3. Mức tăng tích lũy chiết khấu chuẩn hóa tại Top 10 (NDCG@10 – Normalized Discounted Cumulative Gain at 10) [1]: Đánh giá chất lượng xếp hạng tổng thể trong phạm vi 10 kết quả đầu tiên. Chỉ số này phản ánh vị trí ưu tiên: tin nhắn có độ liên quan càng cao mà xếp ở vị trí càng cao thì điểm số tích lũy càng lớn (thông qua hệ số suy giảm chiết khấu logarit $\log_2(i+1)$ theo vị trí thứ hạng i):

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad NDCG@10 = \frac{DCG@10}{IDCG@10} \quad (4.3)$$

Trong đó IDCG@10 là điểm DCG lý tưởng (Ideal DCG) khi các kết quả được xếp hạng tối ưu tuyệt đối theo độ phù hợp giảm dần.

Chỉ số đánh giá (Metric)	Baseline (Standard)	Cốc Cốc + Multi-field	Tăng trưởng
MRR (Mean Reciprocal Rank)	0.6250	0.7708	+23.3%
NDCG@10 (Chất lượng xếp hạng)	0.9701 (nhiều từ rời)	0.9472 (chuẩn ngữ nghĩa)	Chuẩn hóa ngữ nghĩa
Precision@1 (P@1 – Top 1)	58.3%	75.0%	+28.6%
Precision@5 (P@5 – Top 5)	43.3%	56.7%	+30.8%
Độ trễ trung bình (Elasticsearch)	19.0 ms	14.2 ms	Nhanh hơn 25.4%
Độ trễ Custom Go Engine	N/A	3.0 ms	Nhanh hơn 4.7 lần ES

Bảng 4.1: Tổng hợp các chỉ số chất lượng tìm kiếm toàn diện (Macro Benchmark)

#	Truy vấn	Phân loại ngữ nghĩa	MRR		P@5		Độ trễ (Base / Mới)
			Base	Đề xuất	Base	Đề xuất	
1	học sinh	Từ ghép có dấu	1.00	1.00	100%	100%	13ms / 35ms
2	sinh viên	Từ ghép có dấu	1.00	1.00	100%	100%	17ms / 33ms
3	cà phê	Từ ghép có dấu	1.00	1.00	80%	80%	16ms / 25ms
4	phê bình	Từ ghép có dấu	1.00	1.00	40%	40%	18ms / 18ms
5	trà sữa	Từ ghép có dấu	1.00	1.00	60%	60%	16ms / 19ms
6	bàn ghế	Từ ghép có dấu	1.00	1.00	60%	60%	15ms / 21ms
7	bàn bạc	Từ ghép có dấu	1.00	1.00	40%	40%	11ms / 18ms
8	hoc sinh	Không dấu (Unaccented)	0.50	1.00	40%	80%	12ms / 25ms
9	ca phe	Không dấu (Unaccented)	0.00	1.00	0%	80%	12ms / 5ms
10	tra sua	Không dấu (Unaccented)	0.00	0.00	0%	0%	11ms / 11ms
11	ban ghe	Không dấu (Unaccented)	0.00	0.25	0%	40%	13ms / 7ms

Bảng 4.2: Phân tích chi tiết từng kịch bản truy vấn đối chứng (Query-by-Query Analysis)

Tiêu chí kỹ thuật	Elasticsearch 8.11 (Doanh nghiệp)	Custom Go Engine (Tự phát triển)
Dung lượng RAM tối thiểu	1,200 MB (Java Virtual Machine và Docker)	15 MB (nhúng trực tiếp Go)
Độ trễ trung bình	14.2 ms	3.0 ms
Cơ chế ghi bền vững	Translog + Flush Segments [4, 8]	Write-Ahead Log (WAL) + CRC32 IEEE [3]
Khả năng triển khai	Yêu cầu Docker Container, Java Runtime	File nhị phân biên dịch tĩnh duy nhất [7]

Bảng 4.3: So sánh tài nguyên phần cứng giữa các động cơ

4.2 Kỹ năng và kiến thức thu thập được

Qua 12 tuần tham gia chương trình thực tập doanh nghiệp (gồm 6 tuần đào tạo chuyên sâu tại Trường Đại học VinUniversity và 6 tuần thực chiến trực tiếp tại Khối Chat – Công ty Cổ phần Vin Smart Future), em đã tích lũy và hoàn thiện được hệ thống kiến thức chuyên môn cùng các kỹ năng kỹ thuật thực tiễn giá trị:

- Năng lực nghiên cứu lý thuyết Truy xuất thông tin (IR): Nắm vững cơ sở toán học của mô hình xác suất Okapi BM25 [2], cấu trúc Chỉ mục đảo (Inverted Index) và thuật toán duyệt hợp nhất con trỏ Postings [1]. Thấu hiểu sâu sắc bản chất ngôn ngữ đơn lập của tiếng Việt, cơ chế nén từ điển Double-Array Trie (DAT) tra cứu $O(1)$ [10] và thuật toán

- giải mã Viterbi HMM giải quyết nhập nhằng ranh giới từ vựng [5].
2. Kỹ năng lập trình hệ thống cấp thấp và cầu nối liên ngôn ngữ (CGO FFI): Thành thạo kỹ thuật kết nối mã nguồn C++ của Cốc Cốc Tokenizer vào Go Runtime, xử lý hiện tượng Name Mangling qua lớp bọc extern "C", và thiết lập cơ chế giải phóng vùng nhớ C Heap an toàn tuyệt đối với từ khóa defer nhằm triệt tiêu hoàn toàn rò rỉ bộ nhớ (Zero Memory Leak) [5, 6].
 3. Làm chủ công nghệ tìm kiếm phân tán Elasticsearch: Nắm vững kiến trúc lõi Apache Lucene [8], luồng đánh chỉ mục (Indexing Pipeline) và luồng truy vấn (Search Pipeline); thiết kế mô hình dữ liệu đa trường đa tầng và thuật toán Boosting 4 cấp độ [4]; phát hiện và xử lý đứt điểm lỗi đảo ngược điểm số khi tìm kiếm tiền tố gõ dở bằng cấu hình search_analyzer độc lập và toán tử logic AND.
 4. Tư duy thiết kế Động cơ Lưu trữ Nhị phân độc lập (Custom Storage Engine): Làm chủ nguyên lý Log-Structured Merge-Tree (LSM-Tree) [3], cơ chế Write-Ahead Logging (WAL) đảm bảo tính bền vững ACID với mã kiểm tra CRC32 IEEE, tính bất biến của phân đoạn (Segment Immutability) [8], cấu trúc con trỏ Forward Index DocStore tra cứu ngẫu nhiên $O(1)$ qua phép nhân offset cố định 12 bytes trên đĩa, và thuật toán xếp hạng BM25 thuần Go giúp tiết kiệm 91.5% RAM [2, 7].
 5. Phương pháp luận thực nghiệm và tác phong kỹ sư doanh nghiệp: Xây dựng hệ thống đo đạc benchmark tự động đánh giá các chỉ số IR chuẩn quốc tế (MRR, NDCG@10, P@1, P@5) [1]; rèn luyện tư duy phản biện kiến trúc qua các phiên Code Review 1-on-1 cùng Mentor cao cấp; nâng cao năng lực giải quyết bài toán kỹ thuật phức tạp dưới ràng buộc nghiêm ngặt về độ trễ và tài nguyên hệ thống.

4.3 Hướng phát triển tiếp theo để hoàn thiện giải pháp

Từ những kết quả thực nghiệm và kinh nghiệm thu được sau quá trình nghiên cứu, đề tài có thể tiếp tục mở rộng và hoàn thiện theo các định hướng kỹ thuật trọng tâm sau:

1. Tích hợp trực tiếp luồng sự kiện phân tán Kafka (Kafka Event Stream Consumer): Đưa phân hệ tìm kiếm vào vận hành chính thức trên môi trường production, kết nối trực tiếp với cụm Kafka của Khối Chat để tiêu thụ sự kiện tin nhắn thời gian thực với cơ chế kiểm soát áp lực ngược (Backpressure) và xử lý phân tán chịu tải cao.
2. Cơ chế phân mảnh và đồng thuận phân tán (Distributed Sharding & Raft Consensus): Mở rộng Động cơ Lưu trữ Nhị phân thuần Go từ kiến trúc đơn nút (single-node) thành cụm lưu trữ phân tán, phân vùng chỉ mục (Index Sharding) theo mã định danh phòng trò chuyện (room_id) và duy trì tính nhất quán dữ liệu giữa các bản sao qua thuật toán đồng thuận Raft.
3. Mô hình tìm kiếm lai kết hợp ngữ nghĩa sâu (Hybrid Dense-Sparse Retrieval): Tích hợp

mô hình ngôn ngữ tiếng Việt (như PhoBERT hoặc Bi-Encoder Embeddings) để sinh vector ngữ nghĩa, kết hợp song song với thuật toán so khớp từ khóa Okapi BM25 thông qua giải thuật dung hợp thứ hạng Reciprocal Rank Fusion (RRF), giúp giải quyết các câu truy vấn phức tạp hoặc diễn đạt đồng nghĩa.

4. Mô-đun tiền xử lý văn hóa chat (Teencode, từ viết tắt và sửa lỗi gõ phím): Xây dựng bộ từ điển quy đổi tự động và chuẩn hóa các dạng viết tắt, tiếng lóng phổ biến trên ứng dụng nhắn tin ("ko" → "không", "đc" → "được", "ng" → "người", "j" → "gì"), kết hợp giải thuật khoảng cách chỉnh sửa (Edit Distance) để tự động sửa lỗi gõ dính phím do bộ gõ Telex/VNI trước khi đưa vào luồng tìm kiếm.

TÀI LIỆU THAM KHẢO

- [1] Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze (2008), *Introduction to Information Retrieval*, Cambridge University Press, New York, USA.
- [2] Stephen Robertson, Hugo Zaragoza (2009), "The Probabilistic Relevance Framework: BM25 and Beyond", *Foundations and Trends in Information Retrieval*, Vol. 3, No. 4, pp. 333–389.
- [3] Patrick O’Neil, Edward O’Neil, Gerhard Weikum (1996), "The Log-Structured Merge-Tree (LSM-Tree)", *Acta Informatica*, Vol. 33, Issue 4, pp. 351–385.
- [4] Clinton Gormley, Zachary Tong (2015), *Elasticsearch: The Definitive Guide - A Distributed Real-Time Search and Analytics Engine*, O’Reilly Media, Sebastopol, CA, USA.
- [5] Cốc Cốc Team (2018), *Cốc Cốc Tokenizer: C++ High Performance Vietnamese Word Segmentation Library*, Truy cập từ mã nguồn mở GitHub: <https://github.com/coccoc/coccoc-tokenizer>.
- [6] The Go Authors (2024), *Cgo: Foreign Function Interface for Go and C*, Tài liệu kỹ thuật chính thức Golang: <https://go.dev/blog/cgo>.
- [7] Alan A. A. Donovan, Brian W. Kernighan (2015), *The Go Programming Language*, Addison-Wesley Professional, Boston, MA, USA.
- [8] Apache Lucene Project (2024), *Apache Lucene Core Architecture, Inverted Index Formats and Segment Immutability Specification*, Apache Software Foundation.
- [9] Gerard Salton, Michael J. McGill (1983), *Introduction to Modern Information Retrieval*, McGraw-Hill, New York, USA.
- [10] Jun-ichi Aoe (1989), "An Efficient Implementation of Trie Structures", *Software: Practice and Experience*, Vol. 19, No. 11, pp. 1069–1081.



PHIẾU ĐÁNH GIÁ SINH VIÊN MÔN THỰC TẬP DOANH NGHIỆP

1. Thông tin đề tài

- Dự án: GSM Merchant Agent
- Sinh viên thực hiện: Nguyễn Văn Minh

2. Đánh giá

- Số lần / số buổi làm việc trực tiếp: 42 buổi
- Hình thức trao đổi: ☐ Gặp trực tiếp ☐ Trực tuyến (Online) ☐ Email
- Mức độ chủ động: ☐ Rất tốt ☐ Tốt ☐ Trung bình ☐ Kém
- Xác định vấn đề và mục tiêu rõ ràng: .../10
- Quá trình làm việc, bám sát tiến độ: .../10
- Năng lực kỹ thuật và phương pháp triển khai: .../10
- Chất lượng sản phẩm / báo cáo: .../10
- Thái độ học tập, kỷ luật và phản hồi góp ý: .../10

Điểm tổng kết: .../10

3. Nhận xét chung

.....

.....

.....

XÁC NHẬN CỦA DOANH NGHIỆP

(Ký, ghi rõ họ tên và đóng dấu)

NGƯỜI HƯỚNG DẪN

(Ký, ghi rõ họ tên)



Ý kiến đánh giá:

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: Điểm chữ:

Hà Nội, ngày tháng năm 20 .
Giảng viên đánh giá
(Ký, ghi rõ họ tên)